

CubbyHole — Technical Architecture Report

Upload Pipeline, Storage Integrations & Viewer Layer

Prepared: 2026-08-24 **Scope:** Codebase audit of `src/`, `supabase/migrations/`, and root-level `*_schema.sql` files. **Method:** Direct source inspection (no assumptions carried over from prior design docs) — every claim below is traceable to a specific file and line range.

Section 1: Executive Summary & System Overview

CubbyHole is a Next.js 16 / React 19 application for capturing and archiving children’s creations — 360° object scans, textured relief captures, flat 2D artwork, and scanned documents — and storing them as shareable “capsules” of memories.

Confirmed stack:

Layer	Technology
Framework	Next.js 16.2 (App Router), React 19
Styling	Tailwind CSS 4
Cloud backend	Supabase (Postgres + Auth + Storage)
3D rendering	@react-three/fiber 9, @react-three/drei 10, three 0.176
Client-side image processing	browser-image-compression
Local cache	Browser IndexedDB (hand-rolled wrapper, no library)

Important correction to scope: the request asked this report to cover “Supabase, Google Drive, OneDrive, etc.” integrations. A full-repository search for `google.*drive`, `onedrive`, `dropbox`, `googleapis`, and `graph.microsoft` returned **zero matches**. CubbyHole has no third-party cloud storage integration of any kind today — Supabase Storage is the only asset store. The rest of this report does not speculate about unbuilt integrations; Section 5 notes this as an open item only where it’s relevant to sizing future work.

The single most important architectural finding, expanded fully in Section 4, is that **two incompatible data models coexist in this repository**:

- A fully-designed, RLS-complete **multi-household model** (`households` → `children` → `capsules` → `household_links` / `object_permissions` / `one_off_shares`), captured in `supabase/migrations/001_initial_schema.sql`, `002_rls_policies.sql`, and typed in `src/types/database.ts`.
- The **model actually used by every live page** — a simpler `profiles` → `capsules(profile_id)` → `captures(capsule_id)` shape that has **no committed CREATE TABLE migration anywhere in this repository**. It’s only ever referenced through `ALTER TABLE` scripts that assume it already exists.

Every other section below should be read with this split in mind — it explains several of the inconsistencies documented in the Upload Pipeline and Viewer sections.

Section 2: The Upload & Storage Pipeline

2.1 End-to-end flow

A capture goes through **two separate Supabase Storage uploads**, not one:

```

CaptureScreen.tsx (compileScan3D / finishDocument / compileRelief)
└─ uploadCaptureToStorage(primaryBlob)
    └─ supabase.storage.from('raw_captures').upload(...) ← upload #1 (preview only)
        → returns a public URL used ONLY as CapturedMedia.url (NamingScreen preview)

CaptureFlow.tsx (runUpload, after the user confirms naming/metadata)
└─ uploadManager.uploadCapture({ mode, asset, mediaType, pages, frames, reliefFrames })
    └─ POST /api/upload (multipart FormData)
        └─ src/app/api/upload/route.ts
            └─ supabase.storage.from('capsule-assets').upload(...) ← upload #2 (the real one)
                → cloudUrl / cloudPages / cloudFrames / cloudReliefFrames

CaptureFlow.tsx (continued)
└─ saveCapture(...) → local IndexedDB (src/lib/captureDB.ts)
└─ supabase.from('captures').insert(...) → Postgres row referencing upload #2's URLs only

```

upload #1's blob is never referenced again. `capturedMedia.url` (the `raw_captures` URL) is used exclusively as the `<img>` / `<video>` preview source on the `NamingScreen`, then discarded. The row written to the `captures` table always points at the `capsule-assets` copy from upload #2. There is no cleanup job or `storage.remove()` call for the `raw_captures` bucket anywhere in the codebase — every capture leaves an orphaned, uncompressed copy of its primary asset in `raw_captures` permanently. See Section 5 for the cost/complexity implication.

2.2 Client-side packaging (`src/lib/uploadManager.ts`)

- `compressImage()` (lines 20–27) runs every still-image blob through `browser-image-compression` (`maxSizeMB: 0.8`, `maxWidthOrHeight: 1920`, web worker enabled), with a try/catch fallback to the original blob on compression failure.
- Video blobs (`mediaType === 'video'`) are explicitly **not** compressed — passed through unchanged (line 38).
- `uploadCapture()` (lines 29–56) compresses the primary asset plus three optional arrays in parallel (`pages`, `frames`, `reliefFrames` — corresponding to document pages, scan3d's 8 frames, and relief180's up-to-6 frames), then packs everything into a single `FormData` payload keyed as `pages[0]`, `pages[1]`, `frames[0]`, etc., and POSTs to `/api/upload`.

2.3 Server-side endpoint (`src/app/api/upload/route.ts`)

- Single bucket: `capsule-assets` (line 6), `dynamic = 'force-dynamic'` (no caching).
- Reconstructs the indexed arrays back out of `FormData` by probing `pages[0]`, `pages[1]`, ... until a key is missing (lines 44–55) — a manual re-implementation of what `FormData.getAll()` would otherwise do more directly.
- Every file lands under a shared prefix: `${mode}/${timestamp}-${random}` (line 39–40), suffixed `-asset`, `-page-N`, `-frame-N`, `-relief-N`.
- All uploads run via `Promise.all` (lines 59–64) — **all-or-nothing**: if any single frame fails, the whole request throws before any DB row is written, so there's no path to a partial/corrupted asset set reaching Postgres. This is a genuinely good design choice worth preserving.
- Uses the **legacy untyped client** from `src/lib/supabase.ts` (`import { supabase } from '@lib/supabase'`, line 2) rather than the SSR-aware server client in `src/lib/supabase/server.ts`. See Section 5.2 for why this matters.
- Response shape: `{ ok, cloudUrl, sizeBytes, cloudPages?, cloudFrames?, cloudReliefFrames? }` — the multi-frame arrays are only included in the JSON if non-empty (lines 73–75).

2.4 Persistence after upload (`src/components/capture/CaptureFlow.tsx`, `runUpload`, lines 91–185)

Two writes happen after a successful `/api/upload` call:

1. `saveCapture()` into IndexedDB (`src/lib/captureDB.ts`) — always attempted, errors swallowed (`.catch(() => {})`, line 129).
2. `supabase.from('captures').insert(...)` — only if a `capsuleId` was resolved (from the naming screen's capsule picker, or the flow's initial prop). Notably, this insert has a defensive retry:

```
let { error: insertError } = await supabase
  .from('captures')
  .insert({ ...baseRow, ...frameFields, ...timeField })

// Frame/time columns missing from schema (migration not run yet) – retry
// without them so the capture still saves. They can be added later.
if (insertError?.message?.includes('schema cache')) {
  const retry = await supabase.from('captures').insert(baseRow)
  insertError = retry.error
}
```

This comment is itself first-class evidence of the schema-drift problem in Section 4 — the application code cannot assume migrations 003 / 004 have actually been applied to whichever Supabase project it’s talking to, so it degrades gracefully instead of failing outright.

2.5 Local cache layer (src/lib/captureDB.ts)

A hand-rolled IndexedDB wrapper (cubbyhole DB, captures object store, currently version 2). Exposes saveCapture , getAllCaptures , clearCaptures , deleteCapture , updateCapture . Only saveCapture and clearCaptures are ever called anywhere in the app (both from CaptureFlow.tsx) — getAllCaptures , deleteCapture , and updateCapture have zero call sites. In its current wiring this store is write-only: it accumulates a local shadow copy of every capture on save, and can be wiped wholesale via “Clear cache” on the result screen, but nothing ever reads it back. See Section 5.4.

Section 3: Viewer Components & State Management

3.1 Three parallel top-level gallery surfaces

Route	Component	Data source	Notes
/	DashboardGallery → CapsuleDashboard (src/components/dashboard/)	capsules where profile_id = uid , then captures where capsule_id in (...)	Flat “My Memories” grid, ignores capsule grouping entirely
/dashboard	DashboardPage (src/app/dashboard/page.tsx)	capsules where profile_id = uid ; separately, captures grouped by capsule_id for counts/size	Capsule folder list (grid/list toggle, sort, rename/color/delete); defines its own local CapsuleCard — a different component from src/components/dashboard/CapsuleCard.tsx
/dashboard/[id]	DashboardPage (src/app/dashboard/[id]/page.tsx)	captures where capsule_id = id	The actual per-capsule gallery: CaptureCard grid + CaptureViewerModal

`/` and `/dashboard` are functionally overlapping — both are “list of my stuff” screens fetching largely the same tables through slightly different query shapes, wired to two independently-maintained card/skeleton/empty-state component sets. `Middleware` (`src/lib/supabase/middleware.ts`) treats both `/` and any `/dashboard*` path as auth-protected, so both are reachable in the current build.

3.2 Selection state & modals

- `/dashboard/[id]` manages selection through plain local `useState` — `isSelectMode`, a `Set`-backed multi-select (implied by the bulk delete/share affordances in `CaptureCard`’s props), plus `renameTarget` / `colorTarget` / `deleteTarget` single-item state on the parent `/dashboard` page, each rendering a dedicated modal (`RenameCapsuleModal`, `ChangeColorModal`, `DeleteCapsuleModal`) gated by `{target && <Modal .../>}`.
- `CaptureViewerModal.tsx` (`src/components/capture/`) is the single detail/lightbox modal for an individual capture, routed by `capture.mode`:

```
mode === 'scan3d'    → SpinSequenceViewer (default) or ThreeViewer (opt-in toggle)
mode === 'relief180' → LenticularViewer   (default) or ThreeViewer (opt-in toggle)
mode === 'artwork2d' → DocumentViewer
mode === 'document'  → DocumentViewer
mediaType === 'video' → VideoCaptureViewer
```

`ThreeViewer` (`WebGL`, `@react-three/fiber`) is explicitly gated as “Experimental WebGL alt-view, opt-in via a toggle” (comment at `CaptureViewerModal.tsx:17`) and only offered when `hasSpinFrames` / `hasReliefFrames` (≥ 2 frames) — the default experience remains the lighter `SpinSequenceViewer` / `LenticularViewer` CSS/canvas-based viewers. A second dynamic import, `TimeCapsuleViewer` (`src/components/3d/TimeCapsuleViewer.jsx`), is imported at the top of `CaptureViewerModal.tsx` (line 12–15) but **never referenced anywhere else in the file** — a dead import for a component that otherwise has no callers in the whole `src/` tree. It appears to be an earlier, superseded 3D viewer left in place after `ThreeViewer.tsx` replaced it.

- `ScanResultViewer.tsx` is the equivalent “just captured” viewer shown immediately after a capture completes (before it’s necessarily saved to a capsule), and separately imports `ReliefViewer.tsx` for the relief180 case.

3.3 Caching / prop drilling

- No client-side data-fetching library (no React Query / SWR) — every gallery component owns its own `useState` + `useEffect` + direct `supabase.from(...)` calls, with `isLoading` flags managed by hand.
- Cross-component refresh is done by **remount-via-key**: `DashboardGallery` bumps a `galleryKey` integer to force-remount `CapsuleDashboard` after a capture completes (`DashboardGallery.tsx:11-21`); `/dashboard` instead re-runs `fetchCapsules` directly in its `handleCaptureComplete` callback. Two different re-fetch strategies for the same underlying event.
- Thumbnails are rendered as plain `<img src={cloud_url}>` with `onError` → local `imgError` state swapping in a “CLOUD” placeholder tile (`CapsuleDashboard.tsx`’s `MemoryCard`, `dashboard/[id]/page.tsx`’s `CaptureCard`) — no CDN-level resize/transform, no `next/image`, no blur placeholder; every list is fetching full-resolution Supabase Storage URLs directly.
- `NamingScreen`, `CaptureFlow`, and `CaptureScreen` pass capture state down through explicit props and refs (`capturedMediaRef`, `pendingMetadataRef` in `CaptureFlow.tsx`) rather than context — appropriate given the shallow, linear step machine (`capture` → `naming` → `uploading` → `processing` → `result`), but worth knowing if this flow grows a step.

Section 4: Database & Data Schemas

4.1 Model A — versioned, RLS-complete, not used by any live route

Defined in `supabase/migrations/001_initial_schema.sql` + `002_rls_policies.sql`, typed in full in `src/types/database.ts`.

Table	Purpose	Key columns
households	Primary account group (family unit)	owner_user_id → auth.users
children	A child, belongs to a household	household_id, birth_year, avatar_url
capsules	A 3D-object/memory record	household_id, child_id, visibility_tier, object_url, thumbnail_url
household_links	Approved cross-household connections	requester_household_id, recipient_household_id, status
object_permissions	Per-item grant for 'custom' visibility tier	capsule_id, grantee_household_id
one_off_shares	Expiring guest-view tokens	token, expires_at, max_views, view_count

visibility_tier is a 3-state enum (private | linked | custom) with a **4-condition RLS SELECT policy** on capsules (002_rls_policies.sql:76-120): own household, active household_links for 'linked', explicit object_permissions row for 'custom', and one-off token access (D) handled application-side. This is a materially more sophisticated sharing model than anything currently reachable from the UI.

A matching, fully-built-but-unwired React component set exists for exactly this model:

src/components/dashboard/CapsuleCard.tsx, ProfileSelector.tsx, src/components/panels/LinkHouseholdPanel.tsx, src/components/modals/ShareSettingsModal.tsx, src/components/ui/PrivacyBadge.tsx, and their shared types in src/types/dashboard.ts. Confirmed via repo-wide import search: **none of these six files are imported by any page, layout, or the two live dashboard implementations.** They form a coherent, orphaned second UI layer for the household/visibility-tier model.

4.2 Model B — undocumented, actually used by every live page

No CREATE TABLE captures OR CREATE TABLE profiles exists anywhere in this repository. Their shape can only be reconstructed from the ALTER TABLE scripts and the application code that queries them:

Table	Evidence of columns	Source
profiles	id, is_beta_unlocked, storage_limit_bytes	access_codes_schema.sql:24-26, src/app/dashboard/page.tsx:662-670
capsules	id, profile_id, name, theme_color, created_at	src/app/dashboard/page.tsx:15-21 (local Capsule interface)
captures	id, capsule_id, title, description, capture_date, capture_time, location, creator, cloud_url, type, size_bytes, is_public, share_id, cloud_frames[], cloud_relief_frames[], cloud_pages[], created_at	supabase/migrations/003_captures_frame_columns.sql, 004_captures_time_column.sql, public_sharing_schema.sql, storage_size_schema.sql, src/app/dashboard/[id]/page.tsx:22-45
access_codes	id, code, storage_granted_bytes, max_uses, times_used	access_codes_schema.sql:11-18

This is the schema every shipping page actually depends on: /, /dashboard, /dashboard/[id], CaptureFlow.tsx, CaptureScreen.tsx, /api/verify-code, /api/check-code, and /shared/[shareId].

4.3 Consequences of the split

- `src/types/database.ts` (the generated `Database` type consumed by `createBrowserClient<Database> / createServerClient<Database>`) **only covers Model A**. Every query against `captures`, `profiles`, or `access_codes` — i.e. nearly all of the app's real read/write traffic — is therefore made through **untyped** Supabase clients. This is called out explicitly in-repo: `src/lib/supabase/client.ts:11-16` documents that its exported `supabase` singleton is "Left untyped ... since the Database type only covers `capsules` and these call sites also query `captures / profiles`."
- The three `*_schema.sql` files at the repo root (`access_codes_schema.sql`, `public_sharing_schema.sql`, `storage_size_schema.sql`) are explicitly headed **"Review and run manually in the Supabase SQL editor. NOT executed automatically — no CLI/migration runner touched this."** There is no way, from the repository alone, to know which of these have actually been applied to any given Supabase project (staging vs. prod), and the defensive retry logic in `CaptureFlow.tsx` (Section 2.4) exists specifically to survive that uncertainty.
- `public_sharing_schema.sql` additionally documents a **storage-layer/table-layer RLS independence gotcha** worth preserving verbatim (lines 13–23 of that file): `getPublicUrl()` only *constructs* a URL — it does not check permissions — so a capture row correctly marked `is_public = true` can still 403 for anonymous visitors if the `capsule-assets` bucket isn't separately marked public or given its own `storage.objects` RLS policy. The file ships the fix as a commented-out policy, explicitly not yet applied.

4.4 Storage buckets in play

Bucket	Written by	Referenced by	Lifecycle
<code>raw_captures</code>	<code>CaptureScreen.tsx</code> (<code>uploadCaptureToStorage</code>)	Nothing, after the naming screen	No cleanup see Section 2.1 / 5.1
<code>capsule-assets</code>	<code>/api/upload/route.ts</code>	<code>captures.cloud_url / cloud_frames / cloud_pages / cloud_relief_frames</code> , every gallery/viewer	Permanent intended

Section 5: Current Technical Debt & Next Integration Steps

Ordered roughly by impact.

5.1 Every capture uploads its primary asset twice

The `raw_captures` copy (Section 2.1) exists solely to give the naming screen a preview image and is never cleaned up or referenced again. At any real usage volume this is a straightforward, quantifiable storage/egress cost with no product benefit.

Fix options: (a) use `URL.createObjectURL(blob)` for the naming-screen preview instead of a cloud round-trip — it's already a local `Blob` at that point — or (b) delete the `raw_captures` object once the real upload completes. Option (a) is strictly simpler and removes an entire network round-trip from the critical path of every capture.

5.2 Inconsistent Supabase client construction (four distinct patterns)

1. `src/lib/supabase.ts` — legacy `@supabase/supabase-js` anon client, no cookie/session awareness. Used server-side in `/api/upload/route.ts`.
2. `src/lib/supabase/client.ts` — `createClient()` (typed, SSR-cookie-aware) *and* a separate untyped `supabase` singleton, both exported from the same file.
3. `src/lib/supabase/server.ts` — SSR-cookie-aware server client for Server Components.
4. An inline service-role admin client constructed per-request inside `/api/verify-code/route.ts`.

`/api/upload/route.ts` in particular uses pattern (1) — the client with no session context — for a storage write. It currently works because of the bucket's storage policy, but it means the upload endpoint cannot distinguish *which* user is uploading, which will matter the moment per-user storage quotas (already partially modeled via `profiles.storage_limit_bytes`) need enforcing server-side rather than trusted client-side.

5.3 Schema drift between committed migrations and the live application

Sections 4.1–4.3 cover this in full. Concretely, before any further schema work: reconcile `src/types/database.ts` against what the app actually queries (`captures` , `profiles` , `access_codes`), and decide whether Model A (households/visibility tiers/RLS) is a paused-in-progress migration target or dead design — the orphaned component set in 4.1 suggests real work went into it, so this is a “finish or formally shelve” decision, not a cleanup-and-delete one.

5.4 Two redundant/incomplete client-side layers

- IndexedDB cache (`captureDB.ts`) is write-only in practice — `getAllCaptures` / `updateCapture` / `deleteCapture` have no callers. Either wire it into an actual offline-read path (e.g., render the gallery from IndexedDB while Supabase is loading) or remove the unused read/update/delete API surface to stop it looking load-bearing.
- Two top-level “my stuff” surfaces (`/` and `/dashboard`) with separately-maintained card/empty-state/skeleton components fetching overlapping data. Worth an explicit product decision on which one is canonical before either accumulates more feature drift.

5.5 Dead viewer import

`TimeCapsuleViewer` is dynamically imported in `CaptureViewerModal.tsx` but never rendered — safe to remove unless it's mid-swap-in for `ThreeViewer` .

5.6 No cloud storage integration beyond Supabase

As noted in Section 1, Google Drive/OneDrive/Dropbox do not exist in this codebase today. If multi-provider export/sync is on the roadmap, it would be new work, not an extension of an existing partial integration — worth sizing accordingly rather than assuming groundwork is in place. Given Section 5.3's open schema question, resolving *that* first will materially change how much surface a new storage provider needs to touch (e.g., whether `capsules` / `captures` gain a `provider` column now or after the Model A/B decision lands).

5.7 Public sharing bucket policy is explicitly unverified

Per Section 4.3, `public_sharing_schema.sql` ships with its `storage.objects` public-read policy commented out pending manual confirmation of the `capsule-assets` bucket's public/private setting. Anyone enabling public sharing (`/shared/[shareId]`) should confirm this before relying on it — it's the kind of gap that works fine in whichever environment it was last tested in and silently 403s in a fresh one.

Appendix: File Inventory Referenced in This Report

src/app/api/upload/route.ts	– primary upload endpoint (capsule-assets bucket)
src/app/api/verify-code/route.ts	– access-code redemption (service-role client)
src/app/api/check-code/route.ts	– (not detailed above; companion to verify-code)
src/app/page.tsx	– routes to DashboardGallery
src/app/dashboard/page.tsx	– capsule folder list (local CapsuleCard)
src/app/dashboard/[id]/page.tsx	– per-capsule gallery + CaptureViewerModal
src/app/shared/[shareId]/page.tsx	– public/guest capsule viewer
src/lib/supabase.ts	– legacy anon client
src/lib/supabase/client.ts	– SSR browser client (typed + untyped singleton)
src/lib/supabase/server.ts	– SSR server client
src/lib/supabase/middleware.ts	– session refresh + route protection
src/lib/uploadManager.ts	– client-side compression + FormData packaging
src/lib/captureDB.ts	– IndexedDB local cache (write-only in practice)
src/types/database.ts	– generated types for Model A only
src/types/dashboard.ts	– types for the orphaned Model A UI layer
src/components/capture/CaptureFlow.tsx	– capture → naming → upload → processing → result
src/components/capture/CaptureScreen.tsx	– camera capture UI + redundant raw_captures upload
src/components/capture/CaptureViewerModal.tsx	
src/components/capture/ScanResultViewer.tsx	
src/components/capture/SpinSequenceViewer.tsx	
src/components/capture/LenticularViewer.tsx	
src/components/capture/DocumentViewer.tsx	
src/components/capture/VideoCaptureViewer.tsx	
src/components/capture/ReliefViewer.tsx	
src/components/ThreeViewer.tsx	– react-three-fiber experimental viewer
src/components/3d/TimeCapsuleViewer.jsx	– orphaned, dynamically imported but unused
src/components/dashboard/DashboardGallery.tsx	
src/components/dashboard/CapsuleDashboard.tsx	
src/components/dashboard/CapsuleCard.tsx	– orphaned (Model A UI)
src/components/dashboard/ProfileSelector.tsx	– orphaned (Model A UI)
src/components/panels/LinkHouseholdPanel.tsx	– orphaned (Model A UI)
src/components/modals/ShareSettingsModal.tsx	– orphaned (Model A UI)
src/components/ui/PrivacyBadge.tsx	– orphaned (Model A UI)
supabase/migrations/001_initial_schema.sql	
supabase/migrations/002_rls_policies.sql	
supabase/migrations/003_captures_frame_columns.sql	
supabase/migrations/004_captures_time_column.sql	
access_codes_schema.sql	– manual-run only
public_sharing_schema.sql	– manual-run only
storage_size_schema.sql	– manual-run only

PDF Export

This repository has no Markdown→PDF tooling installed today. A minimal script is provided at `scripts/generate-pdf.mjs` (see below) using `markdown-it` + `puppeteer`, both added as dev-only dependencies. It is **not** wired into `package.json`’s scripts by default — run it explicitly:

```
npm install --save-dev puppeteer markdown-it
node scripts/generate-pdf.mjs docs/ARCHITECTURE_REPORT.md docs/ARCHITECTURE_REPORT.pdf
```


This produces a print-formatted PDF with headers, code blocks, and tables styled for legibility. See `scripts/generate-pdf.mjs` for the full implementation.